

Documento de Arquitetura de Software (DAS) v2.2

Lab FinBank & ChefLab — revisão técnica consolidada

Objetivo: fornecer um contrato de implementação suficientemente preciso para Diogo e Erick codificarem os módulos sem depender de decisões implícitas.

Base: DAS v2.1 + lacunas identificadas pela equipe.

Segurança: ambiente acadêmico isolado, somente dados sintéticos, sem exposição pública.

1. Escopo e arquitetura

O laboratório mantém uma aplicação monolítica modular com Node.js + Express + mysql2 e MySQL 8.0. FinBank ocupa /banco/* e demonstra três cenários de SQL Injection; ChefLab ocupa /receitas/* e demonstra XSS armazenado. A seleção do modo é controlada por cookie, e não pela URL das rotas de ataque.

2. Stack tecnológica

Camada	Tecnologia	Contrato
Frontend	HTML5 + CSS/Tailwind + JS	Interfaces FinBank/ChefLab.
Servidor	Node.js + Express	Rotas, middleware e erros.
Acesso a dados	mysql2	Pool/conexões e queries.
Banco	MySQL 8.0	Schema lab_palestra.
Infra	Docker Compose	Inicialização reproduzível.

3. Contrato de modo

Item	Contrato
Cookie	lab_mode
Valores	vuln safe
Escopo	Path=/ middleware/mode.js lê o cookie e define req.labMode
Leitura	
Alteração	POST /api/mode com {"mode":"vuln" "safe"}; servidor responde Set-Cookie
Default no laboratório	vuln, para facilitar a demonstração; a exposição externa deve ser proibida
URL das rotas	Não contém o modo; query string fica reservada aos parâmetros do cenário
Falha/valor inválido	Tratar como vuln no ambiente de laboratório ou rejeitar explicitamente; a escolha deve ser única e documentada

mode.js lê o cookie lab_mode e define req.labMode. POST /api/mode recebe {mode:"vuln"|"safe"} e responde com Set-Cookie. A URL fica reservada aos parâmetros do cenário. O cookie é uma convenção do laboratório, não um mecanismo de segurança.

4. Contrato das rotas

Rota	Vulnerabilidade	Entrada	Modo/Oráculo	Mitigação
/banco/agencias	SQLi UNION	GET cidade	4 colunas na tabela de resultados	Prepared Statement
/banco/extrato	SQLi baseada em erro (didática)	GET id_conta	Erro MySQL exibido somente no vulnerável	Prepared Statement + erro genérico
/banco/noticia	SQLi inferencial/cega	GET id	Notícia / 404 como oráculo	Validação + Prepared Statement
/receitas	XSS armazenado	POST comentário	Comentário persistido	textContent ou sanitização

4.1 /banco/agencias

Modo vulnerável: SELECT de quatro colunas filtradas por cidade. Modo safe: placeholder e connection.execute(sql, [cidade]).

4.2 /banco/extrato

Modo vulnerável: erro didático pode ser mostrado via err.sqlMessage. Modo safe: log interno e mensagem genérica.

4.3 /banco/noticia

Oráculo de conteúdo: notícia versus 404. Erros suprimidos. Safe usa validação complementar e parametrização.

4.4 /receitas

INSERT parametrizado; vulnerabilidade na saída com innerHTML. Safe usa textContent ou sanitização.

5. Dicionário do banco de dados

Este dicionário é parte do contrato e deve ser seguido por init.sql, repositories e rotas.

Tabela	Colunas e tipos	Finalidade / observação
agencias	id_agencia VARCHAR(20) PK; nome_agencia VARCHAR(100); endereco VARCHAR(150); telefone VARCHAR(30); gerente VARCHAR(80); cidade VARCHAR(60)	Filtro por cidade. A consulta pública retorna nome_agencia, endereco, telefone e gerente.
cartoes_credito	id_cartao VARCHAR(20) PK; titular VARCHAR(100); numero_cartao VARCHAR(30); cvv VARCHAR(10); validade VARCHAR(15)	Alvo didático de leitura. Dados exclusivamente fictícios. As 4 colunas podem ser pareadas para o cenário UNION.
administradores	id_admin VARCHAR(20) PK; username VARCHAR(50); senha_hash VARCHAR(100); senha VARCHAR(50)	Artefato didático. id_admin='1' e senha='abcde' são valores de laboratório; senha em texto puro não representa prática real.
clientes	id_cliente VARCHAR(20) PK; nome VARCHAR(100); cpf	Dados complementares, todos sintéticos.

	VARCHAR(20); numero_telefone VARCHAR(30); saldo_conta VARCHAR(30)	
extratos	id_lancamento VARCHAR(20) PK; id_conta VARCHAR(20); descricao VARCHAR(100); valor VARCHAR(30); data_lancamento VARCHAR(20)	Suporte à rota /banco/extrato. Tipos simplificados deliberadamente para o laboratório.
noticias	id VARCHAR(20) PK; titulo VARCHAR(150); conteudo TEXT; data_publicacao VARCHAR(20)	Oráculo booleano da rota /banco/noticia; id='5' é o registro-base.
comentarios_receita	id_comentario INT AUTO_INCREMENT PK; nome_autor VARCHAR(80); texto_comentario TEXT; data_postagem DATETIME	Persistência dos comentários do ChefLab. O risco de XSS está na renderização da saída.

A tabela comentarios_receita não é composta apenas por VARCHAR: usa INT, TEXT e DATETIME. Todos os dados são sintéticos.

6. Cenário 2 — roteiro técnico de teste

Payload previsto para o laboratório, assumindo a consulta vulnerável sobre id_conta:

```
98765' AND EXTRACTVALUE(1, CONCAT(0x7e, (SELECT senha FROM administradores WHERE
id_admin='1')))) --%20
```

O %20 faz o espaço exigido pelo comentário -- chegar ao SQL após a decodificação da URL. A subquery retorna a senha sintética e o resultado é usado na expressão XPath. Com XPath inválido, ExtractValue() pode gerar XPATH syntax error contendo a expressão. A mensagem exata deve ser validada contra a versão MySQL fixada no Docker.

A documentação oficial do MySQL 8.0 registra que locator XPath inválido gera erro. O documento deve distinguir a exposição de erro do cenário completo de SQLi baseada em erro.

7. Reset geral do laboratório

Endpoint	POST /api/reset
Objetivo	Restaurar o estado inicial completo do laboratório.
Banco	Reexecutar seeds/repopular tabelas ou executar rotina transacional equivalente; não depender apenas de DELETE em comentários.
Comentários	Restaurar os dois comentários originais.
Outros dados	Restaurar registros-base de agencias, cartoes_credito, administradores, clientes, extratos e noticias caso tenham sido alterados durante a demonstração.
Restrição	Somente ambiente local/laboratório; não publicar a rota em ambiente externo.
Resposta	204 ou JSON simples de sucesso; não expor detalhes de banco.

Compatibilidade	POST /receitas/reset pode permanecer como alias de laboratório, mas o contrato oficial passa a ser /api/reset.
-----------------	--

POST /receitas/reset pode permanecer como alias, mas o contrato oficial passa a ser POST /api/reset. O reset deve restaurar todo o estado inicial, não apenas comentários.

8. Modo vulnerável × seguro

Componente	Vulnerável	Seguro
SQL	Interpolação controlada	Prepared Statements
Erros	Detalhes didáticos	Log interno + mensagem genérica
XSS	innerHTML	textContent/sanitização
Modo	lab_mode=vuln	lab_mode=safe
Reset	Restaurável	Mesma rotina
Dados	Sintéticos	Sintéticos

Prepared statements são a defesa principal para separar código SQL de valores de entrada.

9. Estrutura de pastas revisada

```
projeto-palestra-seguranca/
├── docker-compose.yml
├── .env.example
├── .gitignore
├── database/init.sql
├── src/server.js
├── src/config/database.js
├── src/middleware/mode.js
├── src/middleware/errorHandler.js
├── src/routes/bancoRoutes.js
├── src/routes/receitasRoutes.js
├── src/routes/labRoutes.js
├── src/repositories/bancoRepository.js
├── src/repositories/receitasRepository.js
├── src/public/banco/
├── src/public/receitas/
├── tests/
└── package.json
```

10. Plano de testes de aceitação

T01	Inicialização	Docker Compose	Aplicação + MySQL iniciam e seeds são carregados.
T02	Modo	POST /api/mode {mode:vuln}	Cookie lab_mode=vuln é criado e rotas entram no modo vulnerável.

T03	Modo	POST /api/mode {mode:safe}	Cookie lab_mode=safe é criado e rotas entram no modo mitigado.
T04	Agências	Consulta normal + cenário UNION	Vulnerável mantém comportamento didático; safe não permite alterar a estrutura lógica da consulta.
T05	Extrato	Payload de erro documentado	Vulnerável produz erro MySQL observável; safe não expõe erro nem altera a query.
T06	Notícia	Condições verdadeira/falsa	Oráculo permanece consistente.
T07	ChefLab	Comentário com marcação HTML	Vulnerável interpreta; safe trata como texto/sanitiza.
T08	Reset	POST /api/reset	Todo o banco volta ao estado inicial.
T09	Isolamento	Tentativa de acesso externo	Serviço não deve ser disponibilizado fora da rede de laboratório.

11. Critérios de aceitação

O clone limpo sobe por Docker Compose e carrega os seeds.

O dicionário de colunas coincide com init.sql e repositories.

POST /api/mode altera o cookie e as quatro rotas respeitam req.labMode.

As URLs permanecem reservadas aos parâmetros dos cenários.

O cenário ExtractValue é reproduzível na versão MySQL fixada ou é ajustado antes da apresentação.

POST /api/reset restaura todo o estado inicial.

O modo seguro usa parametrização e não expõe detalhes de banco.

O XSS é reproduzível no vulnerável e neutralizado no seguro.

O laboratório usa somente dados sintéticos e não é publicado externamente.

12. Referências técnicas

MySQL 8.0 Reference Manual — XML Functions; Prepared Statements; Encryption/Hash Functions.

Esta v2.2 transforma esquema, modo, reset e roteiro do cenário 2 em contratos explícitos de implementação.

2 Lembretes Rápidos para a Hora de Programar (Node.js + mysql2)

Você não precisa alterar mais nada no documento; apenas avise a equipe sobre estes dois detalhes de implementação no Node.js para cumprir os contratos das **Seções 3 e 7**:

1. **Execução do POST /api/reset no mysql2:** Para que a rota /api/reset consiga reexecutar o script database/init.sql inteiro de uma vez (com vários DROP, CREATE e INSERT separados por ponto e vírgula), lembre-se de habilitar a flag multipleStatements: true apenas na função de reset (ou na configuração do src/config/database.js), pois o mysql2 bloqueia múltiplas queries na mesma chamada por padrão.
2. **Leitura do Cookie no src/middleware/mode.js:** Lembre-se de incluir o pacote cookie-parser no package.json (ou fazer um .split(';') simples em req.headers.cookie) para ler lab_mode e definir o fallback único para vuln caso o cookie ainda não exista no navegador do aluno.